МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №4
по дисциплине «ОСНОВЫ МАШИННОГО ОБУЧЕНИЯ»
Тема: «Введение в нейронные сети: построение многослойного перцептрона»

Выполнил:
студент 3 курса
группы АС-67
Доста Е. В.
Проверил:
Крощенко А. А.

2026

Цель работы: построить, обучить и оценить **многослойный перцептрон (MLP)** для решения задачи классификации.

Общий план для всех вариантов:

1. **Импорт библиотек и подготовка данных**
   - импортируйте `torch`, `torch.nn`, `torch.optim`, а также `sklearn` для загрузки данных и их предобработки;
   - загрузите датасет, выполните **стандартизацию** (`StandardScaler`) и **кодирование** признаков;
   - разделите данные на обучающую и тестовую выборки;
   - **преобразуйте данные (признаки и метки) в тензоры PyTorch: `torch.tensor(X_train, dtype=torch.float32)`.**
2. **Определение архитектуры нейронной сети**
   - создайте класс, наследуемый от `torch.nn.Module`;
   - в методе `__init__` определите все слои, которые будете использовать (например, `nn.Linear`, `nn.ReLU`, `nn.Dropout`);
   - в методе `forward` опишите последовательность применения слоев к входным данным.
3. **Инициализация модели, функции потерь и оптимизатора**
   - создайте экземпляр вашей модели: `model = MLP();`
   - определите функцию потерь. Для бинарной классификации используйте `nn.BCEWithLogitsLoss`, для многоклассовой – `nn.CrossEntropyLoss`;
   - определите оптимизатор: `optimizer = torch.optim.Adam(model.parameters(), lr=0.001).`
4. **Написание цикла обучения (Training Loop)**
   - запустите цикл на определенное количество эпох;
   - внутри цикла:
     1. переведите модель в режим обучения: `model.train();`
     2. сделайте предсказание (forward pass): `y_pred = model(X_train);`
     3. рассчитайте потери (loss): `loss = criterion(y_pred, y_train);`
     4. обнулите градиенты: `optimizer.zero_grad();`
     5. выполните обратное распространение ошибки: `loss.backward();`
     6. сделайте шаг оптимизации: `optimizer.step().`
5. **Оценка модели (Evaluation)**
   - переведите модель в режим оценки: `model.eval();`
   - используйте `with torch.no_grad():`, чтобы отключить расчет градиентов;
   - сделайте предсказания на тестовых данных;
   - преобразуйте выходные данные (логиты) в предсказанные классы (например, с помощью `torch.argmax` или проверки порога `> 0`);
   - рассчитайте метрики (`accuracy`, `f1-score` и т.д.), используя `sklearn.metrics`.

Вариант 2 Диагностика рака груди

- Breast Cancer Wisconsin
- **Задача:** определить, является ли опухоль злокачественной (бинарная классификация).
- **Архитектура:**
  - входной слой;
  - один скрытый слой с 16 нейронами (ReLU);
  - выходной слой с 1 нейроном (Sigmoid).
- **Эксперимент:** обучите модель с 32 нейронами в скрытом слое. Как изменились метрики precision и recall?

Код программы:

```
import torch
from torch import nn
from torch import optim
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score,
precision_score, recall_score, f1_score

device = "cuda" if torch.cuda.is_available() else "cpu"
print("Платформа обучения: ", device)

class MLP(nn.Module):
    def __init__(self, in_features: int, hidden: int = 16) -> None:
        super().__init__()
        self.linear_stack = nn.Sequential(
            nn.Linear(in_features=in_features, out_features=hidden),
            nn.ReLU(),
            nn.Linear(in_features=hidden, out_features=1),
        )
    def forward(self, x):
        return self.linear_stack(x)

def train_and_evaluate(hidden: int, data, epochs: int = 200) -> dict:
    X_train_t, X_test_t, y_train_t, y_test_t = data

    torch.manual_seed(42)
    model = MLP(in_features=X_train_t.shape[1], hidden=hidden).to(device)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)
    print(f"\nСкрытый слой: {hidden} нейронов")
    print("\nОбучаемая модель: \n", model)
    print("\tКоличество эпох: ", epochs)
    print("\tОптимизатор(Adam): ")
    for name, val in optimizer.defaults.items():
        print("\t\t", name, " : ", val)
    print("\tФункция потерь(бинарная): ", criterion)
    print("=" * 45)

    for epoch in range(epochs):
        model.train()
        y_pred = model(X_train_t)
        loss = criterion(y_pred, y_train_t)
        optimizer.zero_grad()
        loss.backward()
```

```
        optimizer.step()
        if (epoch + 1) % 50 == 0:
            print(f"Эпоха {epoch + 1}/{epochs}, loss = {loss.item():.4f}")

    print("\nОценка модели нейронной сети: ")
    model.eval()
    with torch.no_grad():
        logits = model(X_test_t)
        predicted_classes = (logits > 0).float()

    y_true = y_test_t.cpu().numpy().ravel()
    y_hat = predicted_classes.cpu().numpy().ravel()

    print(classification_report(
        y_true, y_hat,
        target_names=['доброкачественная (B)', 'злокачественная (M)'],
        zero_division=0,
    ))

    return dict(
        acc=accuracy_score(y_true, y_hat),
        prec=precision_score(y_true, y_hat, zero_division=0),
        rec=recall_score(y_true, y_hat, zero_division=0),
        f1=f1_score(y_true, y_hat, zero_division=0),
    )
def solution() -> None:
    df = pd.read_csv('breast_cancer.csv')
    df = df.loc[:, ~df.columns.str.startswith('Unnamed')].drop(columns='id')
    y = (df['diagnosis'] == 'M').astype(int)
    X = df.drop(columns='diagnosis').astype(float)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.25, random_state=42, stratify=y)
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)
    X_train_t = torch.tensor(X_train_scaled, dtype=torch.float32).to(device)
    X_test_t = torch.tensor(X_test_scaled, dtype=torch.float32).to(device)
    y_train_t = torch.tensor(y_train.values, dtype=torch.float32).view(-1,
1).to(device)
    y_test_t = torch.tensor(y_test.values, dtype=torch.float32).view(-1,
1).to(device)
    data = (X_train_t, X_test_t, y_train_t, y_test_t)
    results = {}
    for hidden in [16, 32]:
        results[hidden] = train_and_evaluate(hidden, data)

    print("\n" + "=" * 45)
    print("Сравнение метрик (16 vs 32 нейрона):")
    print(f"\t{'hidden':>8s} | {'accuracy':>8s} | {'precision':>9s} |
{'recall':>6s} | {'f1':>6s}")
    for hidden, r in results.items():
        print(f"\t{hidden:>8d} | {r['acc']:>8.4f} | {r['prec']:>9.4f} | "
              f"{r['rec']:>6.4f} | {r['f1']:>6.4f}")
    print("\nВывод по эксперименту:")
    print(f"\tprecision: 16 -> {results[16]['prec']:.4f}, 32 ->
{results[32]['prec']:.4f}")
    print(f"\trecall   : 16 -> {results[16]['rec']:.4f}, 32 ->
{results[32]['rec']:.4f}")

if __name__ == "__main__":
    solution()
```

Результат программы:

```
Платформа обучения:  cpu

Скрытый слой: 16 нейронов

Обучаемая модель:
 MLP(
  (linear_stack): Sequential(
    (0): Linear(in_features=30, out_features=16, bias=True)
    (1): ReLU()
    (2): Linear(in_features=16, out_features=1, bias=True)
  )
)
    Количество эпох:  200
    Оптимизатор(Adam):
         lr  :  0.001
         betas  :  (0.9, 0.999)
         eps  :  1e-08
         weight_decay  :  0
         amsgrad  :  False
         maximize  :  False
         foreach  :  None
         capturable  :  False
         differentiable  :  False
         fused  :  None
         decoupled_weight_decay  :  False
    Функция потерь(бинарная):  BCEWithLogitsLoss()
================================================
```

```
Эпоха 50/200, loss = 0.3666
Эпоха 100/200, loss = 0.2133
Эпоха 150/200, loss = 0.1479
Эпоха 200/200, loss = 0.1150

Оценка модели нейронной сети:
                     precision    recall  f1-score   support

доброкачественная (B)       0.98      0.99      0.98        90
  злокачественная (M)       0.98      0.96      0.97        53

            accuracy                           0.98       143
           macro avg       0.98      0.98      0.98       143
        weighted avg       0.98      0.98      0.98       143


Скрытый слой: 32 нейронов
```

```
Обучаемая модель:
 MLP(
  (linear_stack): Sequential(
    (0): Linear(in_features=30, out_features=32, bias=True)
    (1): ReLU()
    (2): Linear(in_features=32, out_features=1, bias=True)
  )
)
    Количество эпох:  200
    Оптимизатор(Adam):
         lr  :  0.001
         betas  :  (0.9, 0.999)
         eps  :  1e-08
         weight_decay  :  0
         amsgrad  :  False
         maximize  :  False
         foreach  :  None
         capturable  :  False
         differentiable  :  False
         fused  :  None
         decoupled_weight_decay  :  False
    Функция потерь(бинарная):  BCEWithLogitsLoss()
================================================
```

```
Эпоха 50/200, loss = 0.3030
Эпоха 100/200, loss = 0.1565
Эпоха 150/200, loss = 0.1062
Эпоха 200/200, loss = 0.0824

Оценка модели нейронной сети:
                     precision    recall  f1-score   support

доброкачественная (B)       0.98      1.00      0.99        90
  злокачественная (M)       1.00      0.96      0.98        53

            accuracy                           0.99       143
           macro avg       0.99      0.98      0.98       143
        weighted avg       0.99      0.99      0.99       143



================================================
Сравнение метрик (16 vs 32 нейрона):
      hidden | accuracy | precision | recall |     f1
          16 |   0.9790 |    0.9808 | 0.9623 | 0.9714
          32 |   0.9860 |    1.0000 | 0.9623 | 0.9808

Вывод по эксперименту:
    precision: 16 -> 0.9808, 32 -> 1.0000
    recall   : 16 -> 0.9623, 32 -> 0.9623
```

Вывод: в ходе работы была обучена нейросеть для диагностики рака груди. Модель с 16 нейронами показала точность 0.98. При увеличении числа нейронов до 32 точность

немного выросла (до 0.99), а метрика precision достигла максимума (1.0). При этом recall остался без изменений (0.96). Увеличение скрытого слоя немного улучшило общую точность, но не повлияло на полноту модели.